

Mermaid Code for Different Diagrams of Our Project “Omni Brain”

1. Query Processing Pipeline

flowchart TD

```
%%=====
%% USER REQUEST & ORCHESTRATION
%%=====
```

User[User Query]

User --> Session[Authenticate User & Validate Session]

Session --> Supervisor[LangGraph Supervisor]

Supervisor --> Rewrite[Query Rewriter]

Rewrite --> Router{Intent & Task Router}

```
%%=====
%% SQL ANALYTICS PATH
%%=====
```

Router --> SQLAgent[Text-to-SQL Agent]

SQLAgent --> SQLValidator[SQL Validator]

SQLValidator --> SQLDB[(PostgreSQL
Structured Business Data)]

SQLDB --> SQLFormatter[Format Structured Results]

```
%%=====
%% MULTI-MODAL RAG PATH
%%=====
```

Router --> Metadata[Extract Metadata Filters]

Metadata --> Embed[Generate Query Embeddings]

Embed --> Hybrid[Hybrid Retrieval Orchestrator]

Hybrid -->|ANN Search| Qdrant[[Qdrant Vector DB]]

Hybrid -->|FTS + Metadata| MetadataDB[(PostgreSQL
Metadata & Full Text Search)]

Qdrant --> DenseResults[Dense Retrieval Results]

MetadataDB --> SparseResults[Sparse Retrieval Results]

DenseResults --> Reranker[Cross-Encoder Reranker]

SparseResults --> Reranker

Reranker --> SemanticContext[Semantic Context]

%%=====

%% KNOWLEDGE GRAPH PATH

%%=====

Router --> GraphAgent[Knowledge Graph Agent]

GraphAgent --> Neo4j[(Neo4j Graph DB)]

Neo4j --> GraphContext[Entity Relationships & Graph Context]

%%=====

%% CONTEXT SYNTHESIS

%%=====

SQLFormatter --> Fusion

SemanticContext --> Fusion

GraphContext --> Fusion

Fusion[Evidence Fusion Engine]

Fusion --> Citation[Citation Builder]

Citation --> Prompt[Prompt Builder]

Prompt --> Guardrails[NeMo Guardrails]

Guardrails --> LLM[LLM Engine]

LLM --> Response[[Grounded Final Response]]

```

%%=====
%% OBSERVABILITY
%%=====

Response --> QueryHistory[(Query History)]

Response --> RetrievalLogs[(Retrieval Logs)]

Response --> Langfuse[(Langfuse Tracing)]

Response --> Feedback[Capture User Feedback]

```

2. Conceptual ER-Diagram

```

erDiagram
    %%=====
    %% ACCESS & ORGANIZATION (Multi-Tenant Knowledge Platform)
    %%=====

    ROLE ||--o{ USER : assigned_to

    DOMAIN ||--o{ COLLECTION : organizes

    DOMAIN ||--o{ DOCUMENT : contains

    COLLECTION }o--o{ DOCUMENT : groups

    %%=====
    %% KNOWLEDGE BASE & CLASSIFICATION
    %%=====

    DOCUMENT ||--o{ DOCUMENT_VERSION : has

    DOCUMENT ||--o{ DOCUMENT_TAG : classified_by

    TAG ||--o{ DOCUMENT_TAG : labels

    %%=====
    %% MULTI-MODAL DOCUMENT STRUCTURE
    %%=====

```

```

DOCUMENT_VERSION ||--o{ PAGE : contains

PAGE ||--o{ CHUNK : consists_of

PAGE ||--o{ IMAGE : includes

PAGE ||--o{ TABLE_ENTITY : includes

%%=====
%% STRUCTURED KNOWLEDGE SOURCES
%%=====

DOMAIN ||--o{ STRUCTURED_DATA_SOURCE : registers

%%=====
%% USER INTERACTION
%%=====

USER ||--o{ QUERY : submits

QUERY ||--o{ RETRIEVED_CONTEXT : produces

QUERY ||--o{ FEEDBACK : receives

%%=====
%% KNOWLEDGE SOURCES
%%=====

RETRIEVED_CONTEXT }o--o{ CHUNK : contains

RETRIEVED_CONTEXT }o--o{ IMAGE : contains

RETRIEVED_CONTEXT }o--o{ TABLE_ENTITY : contains

RETRIEVED_CONTEXT }o--o{ STRUCTURED_DATA_SOURCE : contains

```

3. Logical Database Schema

```

erDiagram

%%=====

```

%% AUTHENTICATION & AUTHORIZATION

%%=====

ROLE ||--o{ USER : assigned_to

```
ROLE {
  uuid role_id PK
  varchar role_name
  text description
}
```

```
USER {
  uuid user_id PK
  uuid role_id FK
  varchar email
  varchar full_name
  boolean is_active
  timestamp created_at
}
```

%%=====

%% KNOWLEDGE ORGANIZATION

%%=====

DOMAIN ||--o{ COLLECTION : organizes

DOMAIN ||--o{ DOCUMENT : owns

COLLECTION ||--o{ COLLECTION_DOCUMENT : contains

DOCUMENT ||--o{ COLLECTION_DOCUMENT : belongs_to

```
DOMAIN {
  uuid domain_id PK
  varchar name
  text description
}
```

```
COLLECTION {
  uuid collection_id PK
  uuid domain_id FK
  varchar name
  text description
}
```

```
COLLECTION_DOCUMENT {
  uuid collection_id FK
  uuid document_id FK
}
```

%%=====

%% DOCUMENT MANAGEMENT

%%=====

DOCUMENT ||--o{ DOCUMENT_VERSION : has

DOCUMENT ||--o{ DOCUMENT_TAG : classified_by
TAG ||--o{ DOCUMENT_TAG : labels

DOCUMENT {
uuid document_id PK
uuid domain_id FK
varchar title
varchar document_type
varchar language
varchar status
timestamp created_at
timestamp updated_at
}

DOCUMENT_VERSION {
uuid version_id PK
uuid document_id FK
uuid storage_id FK
integer version_number
varchar checksum
varchar processing_status
timestamp uploaded_at
timestamp updated_at
}

TAG {
uuid tag_id PK
varchar tag_name
}

DOCUMENT_TAG {
uuid document_id FK
uuid tag_id FK
}

%%=====

%% STORAGE LAYER

%%=====

STORAGE_OBJECT ||--o{ DOCUMENT_VERSION : stores
STORAGE_OBJECT ||--o{ IMAGE : stores

STORAGE_OBJECT {
uuid storage_id PK
varchar provider
varchar bucket_name
varchar object_key

```
varchar mime_type
varchar checksum
}
```

```
%%=====
%% MULTI-MODAL DOCUMENT STRUCTURE
%%=====
```

```
DOCUMENT_VERSION ||--o{ PAGE : contains
```

```
PAGE ||--o{ CHUNK : consists_of
PAGE ||--o{ IMAGE : includes
PAGE ||--o{ TABLE_ENTITY : includes
```

```
PAGE {
uuid page_id PK
uuid version_id FK
integer page_number
varchar ocr_status
varchar ocr_language
}
```

```
CHUNK {
uuid chunk_id PK
uuid page_id FK
integer chunk_order
text content
integer token_count
}
```

```
IMAGE {
uuid image_id PK
uuid page_id FK
uuid storage_id FK
text caption
}
```

```
TABLE_ENTITY {
uuid table_id PK
uuid page_id FK
text markdown_content
jsonb structured_json
}
```

```
%%=====
%% STRUCTURED DATA SOURCES
%%=====
```

```
DOMAIN ||--o{ STRUCTURED_DATA_SOURCE : registers
```

STRUCTURED_DATA_SOURCE ||--o{ STRUCTURED_TABLE : contains

STRUCTURED_TABLE ||--o{ STRUCTURED_COLUMN : defines

```
STRUCTURED_DATA_SOURCE {  
  uuid datasource_id PK  
  uuid domain_id FK  
  varchar name  
  varchar source_kind  
  timestamp updated_at  
}
```

```
STRUCTURED_TABLE {  
  uuid table_id PK  
  uuid datasource_id FK  
  varchar schema_name  
  varchar table_name  
}
```

```
STRUCTURED_COLUMN {  
  uuid column_id PK  
  uuid table_id FK  
  varchar column_name  
  varchar data_type  
}
```

%%=====

%% QUERY ENGINE & OBSERVABILITY

%%=====

USER ||--o{ QUERY : submits

QUERY ||--o{ RETRIEVED_CONTEXT : produces

QUERY ||--o{ QUERY_TRACE : traced_by

QUERY ||--o{ FEEDBACK : receives

RETRIEVED_CONTEXT ||--o{ CONTEXT_ITEM : contains

CONTEXT_ITEM ||--o| CHUNK : references

CONTEXT_ITEM ||--o| IMAGE : references

CONTEXT_ITEM ||--o| TABLE_ENTITY : references

CONTEXT_ITEM ||--o| STRUCTURED_DATA_SOURCE : references

```
QUERY {  
  uuid query_id PK  
  uuid user_id FK  
  text original_query  
  text rewritten_query
```



```
timestamp created_at
timestamp updated_at
}
```

```
QUERY_TRACE {
uuid trace_id PK
uuid query_id FK
varchar provider
varchar external_trace_id
timestamp created_at
}
```

```
RETRIEVED_CONTEXT {
uuid context_id PK
uuid query_id FK
}
```

```
CONTEXT_ITEM {
uuid item_id PK
uuid context_id FK
uuid chunk_id FK
uuid image_id FK
uuid table_id FK
uuid datasource_id FK
numeric relevance_score
integer rank
}
```

```
FEEDBACK {
uuid feedback_id PK
uuid query_id FK
integer rating
text comments
timestamp created_at
}
```

```
%%=====
%% SYSTEM AUDIT
%%=====
```

```
USER ||--o{ AUDIT_LOG : performs
```

```
AUDIT_LOG {
uuid audit_id PK
uuid user_id FK
varchar action
varchar entity_type
uuid entity_id
timestamp action_timestamp
}
```

4. Overall Database Architecture

flowchart TB

```
%%=====
%% USER AND PRESENTATION
%%=====
```

```
subgraph USER["User and Presentation Layer"]
  U["User"]
  UI["Web UI / Streamlit"]
end
```

```
%%=====
%% APPLICATION
%%=====
```

```
subgraph APP["Application Layer"]
  API["FastAPI Backend"]
end
```

```
%%=====
%% AGENT ORCHESTRATION
%%=====
```

```
subgraph AGENT["Agent Orchestration Layer"]
  LG["LangGraph Supervisor"]
  Router["Intent and Task Router"]
  SQLAgent["Text to SQL Agent"]
  VectorAgent["Hybrid Retrieval Agent"]
  GraphAgent["Knowledge Graph Agent - Future"]
end
```

```
%%=====
%% QUERY UNDERSTANDING
%%=====
```

```
subgraph QUERY["Query Understanding"]
  Rewrite["Query Rewriter"]
  Intent["Intent Detection"]
  Domain["Domain Classification"]
end
```

```
%%=====
%% RETRIEVAL SERVICES
%%=====
```

```
subgraph SERVICE["Retrieval Services"]
  Resolver["Metadata Resolver"]
end
```

```
%%=====
%% POLYGLOT DATABASE LAYER
```

```

%%=====
subgraph STORAGE["Polyglot Database Layer"]
PG[["PostgreSQL\nMetadata\nCatalog\nTransactions\nAudit"]]
QD[["Qdrant\nVector Embeddings"]]
SQLDB[["Structured Data Sources\nText to SQL"]]
MINIO[["MinIO\nObject Storage"]]
NEO[["Neo4j\nKnowledge Graph\nFuture"]]
end

%%=====
%% CONTEXT ASSEMBLY
%%=====
subgraph CONTEXT["Evidence Fusion Layer"]
Merge["Evidence Fusion Engine"]
Citation["Citation Builder"]
end

%%=====
%% REASONING
%%=====
subgraph REASON["Reasoning Layer"]
Guard["NeMo Guardrails"]
LLM["LLM Generation Engine"]
end

%%=====
%% OBSERVABILITY
%%=====
subgraph OBS["Observability"]
Langfuse[["Langfuse"]]
end

%%=====
%% FEEDBACK
%%=====
subgraph FEEDBACK["Feedback and Evaluation"]
FB["User Feedback"]
end

%%=====
%% CLIENT FLOW
%%=====
U --> UI
UI --> API
API --> LG

%%=====
%% QUERY FLOW
%%=====
LG --> Router
Router --> Rewrite
Router --> Intent

```

Router --> Domain

Rewrite --> LG

Intent --> LG

Domain --> LG

%%=====

%% AGENT EXECUTION

%%=====

LG --> SQLAgent

LG --> VectorAgent

LG --> GraphAgent

%%=====

%% DATABASE ACCESS

%%=====

SQLAgent --> SQLDB

VectorAgent --> QD

QD -->|"Vector Matches"| Resolver

Resolver --> PG

PG -->|"Metadata and Document Context"| Merge

SQLDB -->|"Structured Results"| Merge

GraphAgent --> NEO

NEO -->|"Entity Relationships"| Merge

%%=====

%% OBJECT STORAGE

%%=====

PG -->|"Object Keys"| MINIO

API -->|"Authorized Object Access"| MINIO

%%=====

%% RESPONSE GENERATION

%%=====

Merge --> Citation

Citation --> Guard

Guard --> LLM

LLM --> API

API --> UI

%%=====

%% OBSERVABILITY

%%=====

API --> Langfuse

LG --> Langfuse

LLM --> Langfuse

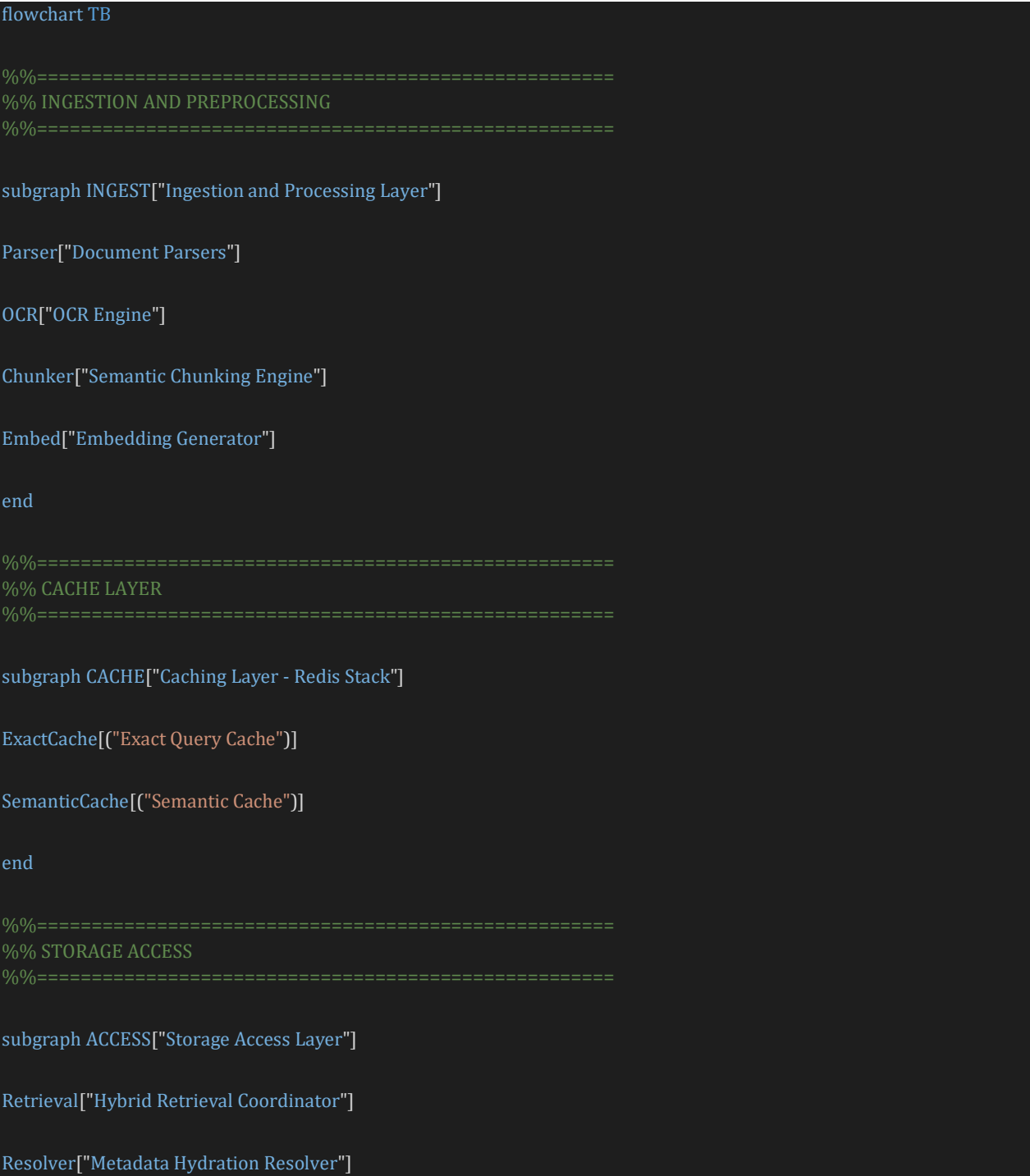
%%=====

%% FEEDBACK LOOP

%%=====

UI --> FB

5. Internal Storage Architecture



```
end

%%=====
%% POLYGLOT STORAGE
%%=====

subgraph STORAGE["Polyglot Storage Layer"]

subgraph PG["PostgreSQL Cluster"]

PG_Meta["Metadata Catalog"]

PG_Doc["Document Hierarchy Tables"]

PG_Audit["Audit and History Logs"]

PG_Index[["Physical Indexes\nB-tree\nGIN\nBRIN\nComposite"]]

end

subgraph VECTOR["Vector Storage"]

QD[["Qdrant Vector Store"]]

end

subgraph OBJECT["Object Storage"]

MINIO[["MinIO Blob Storage"]]

end

end

%%=====
%% CONTEXT ASSEMBLY
%%=====

subgraph CONTEXT["Context Assembly"]

Fusion["Evidence Fusion Engine"]

end

%%=====
```

```
%% INGESTION FLOW
%%=====

Parser -->|"Store Raw Blobs"| MINIO

MINIO -->|"Return Object Keys"| PG_Meta

Parser -->|"Extract Layout Metadata"| PG_Meta

Parser -->|"Route Scanned Pages"| OCR

OCR --> Chunker

Chunker -->|"Write Text Hierarchy"| PG_Doc

Chunker --> Embed

Embed -->|"Ingest Vector Payloads"| QD

PG_Meta --> PG_Doc

%%=====
%% CACHE LOOKUP
%%=====

Retrieval -->|"Check Exact Cache"| ExactCache

Retrieval -->|"Check Semantic Cache"| SemanticCache

%%=====
%% STORAGE ACCESS AND HYDRATION
%%=====

Retrieval -->|"Cache Miss"| QD

QD -->|"Vector Matches and Point IDs"| Resolver

Resolver -->|"Hydrate Relational Context"| PG_Doc

Resolver -->|"Metadata Validation"| PG_Meta

Resolver -->|"Fetch Referenced Media"| MINIO

%%=====
%% CONTEXT BUILDING
%%=====
```

Resolver -->|"Hydrated Context Payload"| Fusion

```
%%=====
%% CACHE REFRESH
%%=====
```

Fusion -->|"Asynchronously Refresh Cache"| ExactCache

Fusion -->|"Asynchronously Refresh Cache"| SemanticCache

```
%%=====
%% INDEX MAINTENANCE
%%=====
```

PG_Meta -->|"Index Maintenance"| PG_Index

PG_Doc -->|"Index Maintenance"| PG_Index

PG_Audit -->|"Index Maintenance"| PG_Index

6. Physical PostgreSQL Schema Architecture

flowchart TB

```
%%=====
%% DATABASE ENGINE
%%=====
DB[["OmniBrain PostgreSQL Database\nCluster: omnibrain_db"]]
```

```
%%=====
%% EXTENSION LAYER
%%=====
subgraph EXT["PostgreSQL Extensions"]
  UUID["uuid-osspl\nUUID Generation Extension"]
  TRGM["pg_trgm\nTrigram Search Extension"]
end
```

```
%%=====
%% AUTHENTICATION SCHEMA
%%=====
subgraph AUTH["Schema: auth"]
  ROLE["roles"]
  USER["users"]
end
```



```

%%=====
%% KNOWLEDGE MANAGEMENT SCHEMA
%%=====
subgraph KNOW["Schema: knowledge"]
DOMAIN["domains"]
COLLECTION["collections"]
DOC["documents"]
DOCVER["document_versions"]
PAGE["pages"]
CHUNK["chunks"]
IMAGE["images"]
TABLEE["table_entities"]
TAG["tags"]
DOCTAG["document_tags"]
COLDOC["collection_documents"]
STORAGE["storage_objects"]
end

%%=====
%% STRUCTURED DATA SCHEMA
%%=====
subgraph STRUCT["Schema: structured"]
DS["structured_data_sources"]
ST["structured_tables"]
SC["structured_columns"]
end

%%=====
%% QUERY ENGINE SCHEMA
%%=====
subgraph QUERY["Schema: query_engine"]
Q["queries"]
QT["query_trace"]
RC["retrieved_context"]
CI["context_item"]
FB["feedback"]
end

%%=====
%% AUDIT SCHEMA
%%=====
subgraph AUD["Schema: audit"]
AL["audit_log"]
end

%%=====
%% PHYSICAL STORAGE INFRASTRUCTURE
%%=====
subgraph IDX["Physical Storage Infrastructure"]
PK["Primary Keys\nImplicit B-tree Indexes"]
FK["Foreign Key Constraints\nRecommended Supporting Indexes"]

```

```

BTREE["B-tree Indexes\nExact Match & Ordered Lookups"]
GIN["GIN Indexes\nSONB • Arrays • Full-Text • Trigram"]
BRIN["BRIN Indexes\nLarge Ordered Datasets"]
CMP["Composite Indexes\nMulti-Column Queries"]
end

%%=====
%% DATABASE ORGANIZATION
%%=====
DB --> EXT
DB --> AUTH
DB --> KNOW
DB --> STRUCT
DB --> QUERY
DB --> AUD

%%=====
%% CROSS-SCHEMA DEPENDENCIES
%%=====
AUTH -->|"Ownership & Access Control"| KNOW
KNOW -->|"Structured Data Registration"| STRUCT
KNOW -->|"Knowledge Context"| QUERY
QUERY -->|"Operational Audit Trail"| AUD

%%=====
%% STORAGE INFRASTRUCTURE
%%=====
KNOW -->|"Physical Storage"| IDX
STRUCT -->|"Physical Storage"| IDX
QUERY -->|"Physical Storage"| IDX
AUD -->|"Physical Storage"| IDX

```